- **Internship Name : Data Analysis**
- **Task Name: Task 10: Data Summarization Using Pandas**
- **Your Name : Shubhangi Naresh Khonde**
- **Internship Organization :**
- **Date:**

1. Objective

To practice filtering rows, sorting data, and summarizing a dataset using the groupby() and agg() functions in Pandas.

2. Tools and Environment

- Language: Python 3
- Library: Pandas
- Environment: Jupyter Notebook in Visual Studio Code

3. Dataset Description

A sample employee dataset with 8 records and 5 columns was created using a Python dictionary and converted into a DataFrame.

Column	Description
Name	Employee name
Department	Sales, HR or IT
City	Nagpur, Pune or Mumbai
Salary	Monthly salary in rupees
Age	Employee age

The dataset was inspected using head(), shape, info() and describe() to understand its structure and data types.

```
##Create DataSet
```

```
data = {
    "Name":       ["Amit", "Sneha", "Rahul", "Priya", "Karan", "Neha", "Rohan", "Anjali"],
    "Department": ["Sales", "HR", "Sales", "IT", "IT", "HR", "IT", "Sales"],
    "City":       ["Nagpur", "Pune", "Nagpur", "Mumbai", "Pune", "Nagpur", "Mumbai", "Pune"],
    "Salary":     [30000, 45000, 28000, 60000, 55000, 42000, 65000, 33000],
    "Age":        [25, 32, 28, 35, 30, 29, 40, 27]
}

df = pd.DataFrame(data)
print(df)
```

```
     Name Department    City  Salary  Age
0    Amit      Sales  Nagpur   30000   25
1   Sneha         HR    Pune   45000   32
2   Rahul      Sales  Nagpur   28000   28
3   Priya         IT  Mumbai   60000   35
4   Karan         IT    Pune   55000   30
5    Neha         HR  Nagpur   42000   29
6   Rohan         IT  Mumbai   65000   40
7  Anjali      Sales    Pune   33000   27
```

4. Methodology

4.1 Filtering Rows

Rows were selected using Boolean conditions:

- Salary greater than 40000: df[df["Salary"] > 40000]

##Step 3: Rows filter

```
# Salary 40000 peksha jast
high_salary = df[df["Salary"] > 40000]
print(high_salary)
```

```
    Name Department    City  Salary  Age
1  Sneha         HR    Pune   45000   32
3  Priya         IT  Mumbai   60000   35
4  Karan         IT    Pune   55000   30
5   Neha         HR  Nagpur   42000   29
6  Rohan         IT  Mumbai   65000   40
```

- Only the IT department: df[df["Department"] == "IT"]

```
# Only IT department
it_dept = df[df["Department"] == "IT"]
print(it_dept)
```

```
    Name Department    City  Salary  Age
3  Priya         IT  Mumbai   60000   35
4  Karan         IT    Pune   55000   30
6  Rohan         IT  Mumbai   65000   40
```

- Multiple conditions using &: df[(df["Department"] == "IT") & (df["Salary"] > 55000)]

```
# Multiple conditions: AND (&) AND OR (|)
result = df[(df["Department"] == "IT") & (df["Salary"] > 55000)]
print(result)
```

```
    Name Department    City  Salary  Age
3  Priya         IT  Mumbai   60000   35
6  Rohan         IT  Mumbai   65000   40
```

- Membership test using isin(): df[df["City"].isin(["Nagpur", "Pune"])]

```
# Nagpur kiwa Pune madhle
cities = df[df["City"].isin(["Nagpur", "Pune"])]
print(cities)
```

```
      Name Department    City  Salary  Age
0    Amit      Sales  Nagpur   30000   25
1   Sneha         HR    Pune   45000   32
2   Rahul      Sales  Nagpur   28000   28
4   Karan         IT    Pune   55000   30
5    Neha         HR  Nagpur   42000   29
7  Anjali      Sales    Pune   33000   27
```

- Using query(): df.query("Age > 28 and Salary < 50000")

```
# query() vaprun pan karta yeta
result2 = df.query("Age > 28 and Salary < 50000")
print(result2)
```

```
    Name Department    City  Salary  Age
1  Sneha         HR    Pune   45000   32
5   Neha         HR  Nagpur   42000   29
```

Observations:

- 5 employees earn more than 40000 (Sneha, Priya, Karan, Neha, Rohan).
- Only Priya and Rohan are in IT with a salary above 55000.
- Only Sneha and Neha satisfy both Age > 28 and Salary < 50000.

4.2 Sorting Data

The sort_values() function was used to arrange the data:

- Ascending by Salary

```
# Salary ascending
df.sort_values("Salary")
print(df.sort_values("Salary"))
```

```
     Name Department    City  Salary  Age
2   Rahul      Sales  Nagpur   28000   28
0    Amit      Sales  Nagpur   30000   25
7  Anjali      Sales    Pune   33000   27
5    Neha         HR  Nagpur   42000   29
1   Sneha         HR    Pune   45000   32
4   Karan         IT    Pune   55000   30
3   Priya         IT  Mumbai   60000   35
6   Rohan         IT  Mumbai   65000   40
```

- Descending by Salary (ascending=False)

```
# Descending
df.sort_values("Salary", ascending=False)
print("\nSalary descending:")
print(df.sort_values("Salary", ascending=False))
```

```
Salary descending:
     Name Department    City  Salary  Age
6   Rohan         IT  Mumbai   65000   40
3   Priya         IT  Mumbai   60000   35
4   Karan         IT    Pune   55000   30
1   Sneha         HR    Pune   45000   32
5    Neha         HR  Nagpur   42000   29
7  Anjali      Sales    Pune   33000   27
0    Amit      Sales  Nagpur   30000   25
2   Rahul      Sales  Nagpur   28000   28
```

- Multiple columns: Department ascending, then Salary descending

```
# Multiple columns: First Department, Then Salary descending
df.sort_values(["Department", "Salary"], ascending=[True, False])
print("\nDepartment (A-Z), mag Salary (jast pasun kami):")
print(df.sort_values(["Department", "Salary"], ascending=[True, False]))
```

[5] ✓ 0.0s

```
Department (A-Z), mag Salary (jast pasun kami):
     Name Department    City  Salary  Age
1   Sneha         HR    Pune   45000   32
5    Neha         HR  Nagpur   42000   29
6   Rohan         IT  Mumbai   65000   40
3   Priya         IT  Mumbai   60000   35
4   Karan         IT    Pune   55000   30
7  Anjali      Sales    Pune   33000   27
0    Amit      Sales  Nagpur   30000   25
2   Rahul      Sales  Nagpur   28000   28
```

- Filtering and sorting combined (IT employees by highest salary)

```python
# Filter + sort ekatra
top_it = df[df["Department"] == "IT"].sort_values("Salary", ascending=False)
print("\nIT department, Salary descending:")
top_it = df[df["Department"] == "IT"].sort_values("Salary", ascending=False)
print(top_it)
```

```
IT department, Salary descending:
    Name Department    City  Salary  Age
6  Rohan         IT  Mumbai   65000   40
3  Priya         IT  Mumbai   60000   35
4  Karan         IT    Pune   55000   30
```

Observation: Rohan (IT, 65000) has the highest salary and Rahul (Sales, 28000) has the lowest. Sorting returns a new DataFrame and does not change the original df.

4.3 Grouping and Aggregation

The groupby() function was used with agg() to compute summary statistics per department:

```
python
df.groupby("Department").agg(
   total_employees=("Name", "count"),
   avg_salary=("Salary", "mean"),
   max_salary=("Salary", "max"),
   min_salary=("Salary", "min"),
   total_salary=("Salary", "sum")
)
```

```python
# Simple: department-wise average salary
df.groupby("Department")["Salary"].mean()

# Multiple aggregations ekatra
summary = df.groupby("Department").agg(
    total_employees=("Name", "count"),
    avg_salary=("Salary", "mean"),
    max_salary=("Salary", "max"),
    min_salary=("Salary", "min"),
    total_salary=("Salary", "sum")
)
print(summary)
```

```
            total_employees    avg_salary  max_salary  min_salary  \
Department
HR                        2  43500.000000       45000       42000
IT                        3  60000.000000       65000       55000
Sales                     3  30333.333333       33000       28000

            total_salary
Department
HR                 87000
IT                180000
Sales              91000
```

Grouping by two columns (Department and City) was also performed to get a more detailed breakdown.

4.4 Final Combined Summary

Filtering, grouping, and sorting were combined into one pipeline:

```
python
filtered = df[df["Age"] > 25]

summary = filtered.groupby("Department").agg(
   employees=("Name", "count"),
   avg_salary=("Salary", "mean"),
   max_age=("Age", "max")
).reset_index()

summary = summary.sort_values("avg_salary", ascending=False)
print(summary)
```

Output:

Department	Employees	Avg Salary	Max Age
IT	3	60000.0	40
HR	2	43500.0	32
Sales	2	30500.0	28

```python
# 1. Filter: fakt 25+ age che employees
filtered = df[df["Age"] > 25]

# 2. Group + aggregate
summary = filtered.groupby("Department").agg(
    employees=("Name", "count"),
    avg_salary=("Salary", "mean"),
    max_age=("Age", "max")
).reset_index()

# 3. Sort: avg salary jast pasun kami
summary = summary.sort_values("avg_salary", ascending=False)

print(summary)
```

```
  Department  employees  avg_salary  max_age
1         IT          3     60000.0       40
0         HR          2     43500.0       32
2      Sales          2     30500.0       28
```

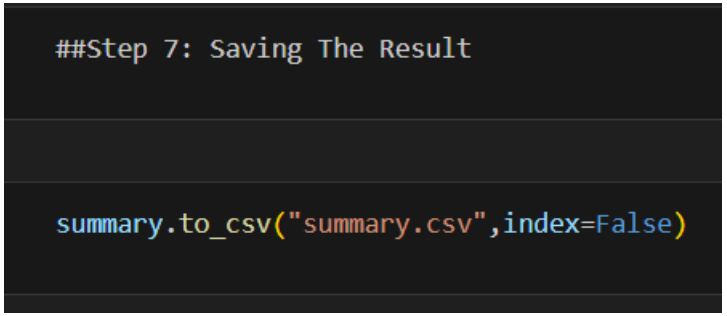
4.5 Saving the Result

The final summary was exported to a CSV file:

python

summary.to_csv("summary.csv", index=False)

The parameter index=False prevents the row index from being written as an extra column.



5. Key Findings

- The IT department has the highest average salary (60000) and the oldest employee (age 40).
- The Sales department has the lowest average salary (30500).
- HR sits in the middle, with an average salary of 43500.
- After removing employees aged 25 or below, Sales drops from 3 to 2 employees.

6. Conclusion

This task demonstrated how Pandas can be used to filter rows with Boolean conditions, sort data on one or more columns, and summarize data with groupby() and agg(). Combining these operations into one workflow gives a clear, sorted summary of a dataset, which was exported to summary.csv. These techniques are the foundation of real-world data analysis.